

ĐẠI HỌC BÁCH KHOA HÀ NỘI
Trường Công nghệ Thông tin và Truyền thông

BÁO CÁO ĐỒ ÁN

Hệ thống định danh người dùng qua hành vi sử dụng
điện thoại

Môn học: Nhập môn Khoa học dữ liệu
Hướng tiếp cận: Random Forest, SVM (RBF) và CNN 1D
Dữ liệu: Hành vi cảm biến điện thoại và gõ phím

Thành viên nhóm

Lương Mạnh Tường	20225949
Bùi Minh Tuấn	20225678
Trương Quốc Triệu	20225941
Nguyễn Đức Thành	20225930
Nguyễn Anh Tuấn	20225772
Vũ Ngọc Lâm	20225645
Vũ Nhật Quang	20225761
Mai Văn Đăng	20225699
Nguyễn Đình Thành	20225670

Mục lục

Tóm tắt	1
1 Mô tả bài toán thực tế	2
1.1 Bối cảnh	2
1.2 Phát biểu bài toán	2
1.3 Ý nghĩa thực tế	2
2 Phương pháp học máy và dataset sử dụng	2
2.1 Dataset	2
2.2 Tiền xử lý dữ liệu cảm biến	3
2.3 Cách chia dữ liệu	5
2.4 Mô hình Random Forest	6
2.5 Mô hình SVM (RBF)	7
2.6 Mô hình CNN 1D	7
3 Kết quả thí nghiệm đánh giá hiệu năng	8
3.1 Chỉ số đánh giá	8
3.2 Kết quả tổng thể	9
3.3 Độ ổn định theo fold	10
3.4 Accuracy của Random Forest	11
3.5 Phân tích mô hình Random Forest	11
3.6 Phân tích CNN theo hoạt động	14
4 Các chức năng chính của hệ thống và cách sử dụng	16
4.1 Chức năng chính	16
4.2 Cài đặt môi trường	16
4.3 Chạy pipeline đơn giản	16
4.4 Chạy bằng JupyterLab	17
5 Cấu trúc mã nguồn và vai trò các class/method chính	17
5.1 Tổ chức mã nguồn	17
5.2 Vai trò các module chính	17
5.3 Class và method quan trọng	18
6 Khó khăn gặp phải và cách giải quyết	18

6.1	Tên file và schema dữ liệu không đồng nhất	18
6.2	Nguyên cơ rò rỉ dữ liệu khi chia train/validation	18
6.3	Ít file ở một số người dùng	19
6.4	Mất cân bằng số cửa sổ giữa các file	19
6.5	Dữ liệu cảm biến có nhiễu và khác tần số	19
6.6	CNN không vượt Random Forest	19
7	Khám phá mới hoặc kết luận	19
7.1	Khám phá chính	19
7.2	Kết luận	20
7.3	Hướng phát triển	20
7.4	Đóng gói sản phẩm nội	20

Tóm tắt

Đồ án khảo sát bài toán định danh người dùng từ dữ liệu sử dụng điện thoại theo chế độ closed-set với 15 người dùng. Phần mô hình chính dùng tín hiệu cảm biến quán tính; log gõ phím được giữ như nguồn dữ liệu phụ để kiểm tra và mở rộng sau. Ba mô hình được so sánh: Random Forest và SVM (RBF) trên 65 đặc trưng thủ công, cùng CNN 1D học trực tiếp trên chuỗi cảm biến. Với cách đánh giá 3-fold *StratifiedGroupKFold* chia theo file để tránh rò rỉ dữ liệu, Random Forest đạt macro-F1 trung bình 0.7434, SVM (RBF) đạt 0.6550, và CNN 1D simple đạt 0.6272. Trên tập dữ liệu hiện tại với số phiên ghi giới hạn cho mỗi người dùng, hai mô hình dùng đặc trưng thống kê thủ công cho macro-F1 cao hơn CNN 1D học đặc trưng tự động.

1 Mô tả bài toán thực tế

1.1 Bối cảnh

Điện thoại cá nhân thường chứa tài khoản, tin nhắn, ảnh và nhiều dữ liệu nhạy cảm. Mật khẩu, vân tay hoặc nhận diện khuôn mặt bảo vệ tốt ở thời điểm mở khoá, nhưng không theo dõi liên tục trong quá trình sử dụng. Sinh trắc học hành vi tiếp cận vấn đề từ một góc khác: nhận ra người dùng qua cách họ cầm, di chuyển và thao tác với thiết bị.

Trong đề án này, hệ thống cần dự đoán người đang sử dụng điện thoại từ dữ liệu hành vi. Nguồn chính là chuyển động của điện thoại khi người dùng ngồi, đứng và đi bộ; dữ liệu gõ phím được dùng để ghi nhận thêm nhịp tương tác.

1.2 Phát biểu bài toán

Với mỗi mẫu dữ liệu cảm biến quán tính có dạng chuỗi thời gian:

$$X_i \in \mathbb{R}^{128 \times 6}$$

trong đó 128 là số mẫu trong một cửa sổ 2.56 giây, 6 kênh gồm `acc_x`, `acc_y`, `acc_z`, `gyro_x`, `gyro_y`, `gyro_z`. Mục tiêu là học hàm phân loại:

$$f(X_i) \rightarrow y_i, \quad y_i \in \{1, 2, \dots, 15\}$$

với y_i là mã của một trong 15 người dùng trong dữ liệu.

Đề án giải bài toán ở chế độ **closed-set**: mọi mẫu kiểm thử đều thuộc về một trong 15 người dùng đã xuất hiện trong huấn luyện. Phạm vi closed-set cho phép tập trung đánh giá khả năng phân biệt người dùng giữa các mô hình trên cùng một tập lớp cố định. Bài toán phát hiện người dùng lạ (open-set) nằm ngoài phạm vi đề án vì đòi hỏi thêm dữ liệu người dùng chưa biết và phương pháp ước lượng ngưỡng riêng.

1.3 Ý nghĩa thực tế

Nếu đạt độ tin cậy đủ cao, hướng này có thể dùng cho xác thực liên tục hoặc cảnh báo khi thiết bị có dấu hiệu bị người khác sử dụng. Trong đề án, trọng tâm là xây dựng pipeline dữ liệu có thể chạy lại, chia train/validation đúng cách và giải thích rõ giới hạn của kết quả.

2 Phương pháp học máy và dataset sử dụng

2.1 Dataset

Bộ dữ liệu dùng trong báo cáo gồm 15 người dùng và 185 file CSV sau khi bỏ các log không dùng trong pipeline cuối. Các file còn lại được chia thành hai nhóm:

- **Inertial**: dữ liệu cảm biến quán tính, gồm gia tốc kế và con quay hồi chuyển. Đây là nguồn dữ liệu chính dùng cho mô hình.

- **Keystroke**: dữ liệu gõ phím như thời điểm gõ, khoảng cách thời gian giữa các phím, tốc độ gõ.

Sau bước quét dữ liệu, `manifest.csv` được sinh ra để chuẩn hoá thông tin: người dùng, tên file, modality, hoạt động, session, attempt/rep, số dòng và tần số lấy mẫu ước lượng. Bảng 1 và 2 tóm tắt quy mô dữ liệu sau khi chuẩn hoá manifest.

Bảng 1: Số lượng file theo modality

Modality	Số file
Inertial	154
Keystroke	31
Tổng	185

Bảng 2: Số file inertial theo hoạt động

Hoạt động	Số file inertial
sitting	46
standing	46
walking	62

Mô hình chính dùng inertial vì đây là chuỗi thời gian liên tục và khớp trực tiếp với cả hai hướng mô hình: Random Forest trên đặc trưng thống kê và CNN 1D trên tín hiệu thô đã xử lý. Keystroke được trích đặc trưng trong `features.py` nhưng chưa đưa vào kết quả chính, để phần đánh giá tập trung vào một nguồn tín hiệu rõ ràng. Tần số lấy mẫu thực tế giữa các file không hoàn toàn đồng nhất, vì vậy pipeline có bước resample về 50 Hz trước khi chia cửa sổ.

2.2 Tiền xử lý dữ liệu cảm biến

Pipeline tiền xử lý được cài đặt trong `preprocess.py`. Các bước chính:

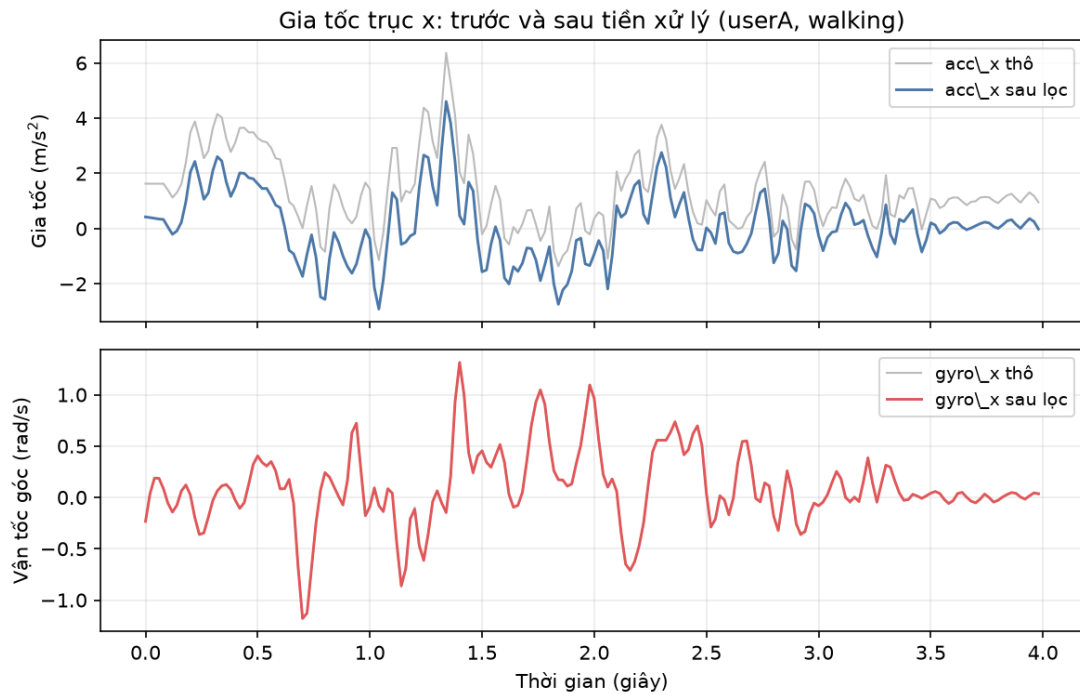
1. Đọc dữ liệu cảm biến từ CSV.
2. Điền giá trị thiếu bằng forward-fill/backward-fill.
3. Resample về 50 Hz nếu tần số lấy mẫu thực tế lệch nhiều.
4. Lọc low-pass Butterworth cutoff 20 Hz để giảm nhiễu cao tần.
5. Lọc high-pass Butterworth cutoff 0.3 Hz cho ba trục gia tốc để tách thành phần trọng lực.
6. Chia cửa sổ 128 mẫu, bước trượt 64 mẫu, tương đương overlap 50%.

- Giới hạn tối đa 300 cửa sổ mỗi file để tránh một file dài chi phối toàn bộ tập huấn luyện.

Kết quả lưu tại `windows.npz` với tensor:

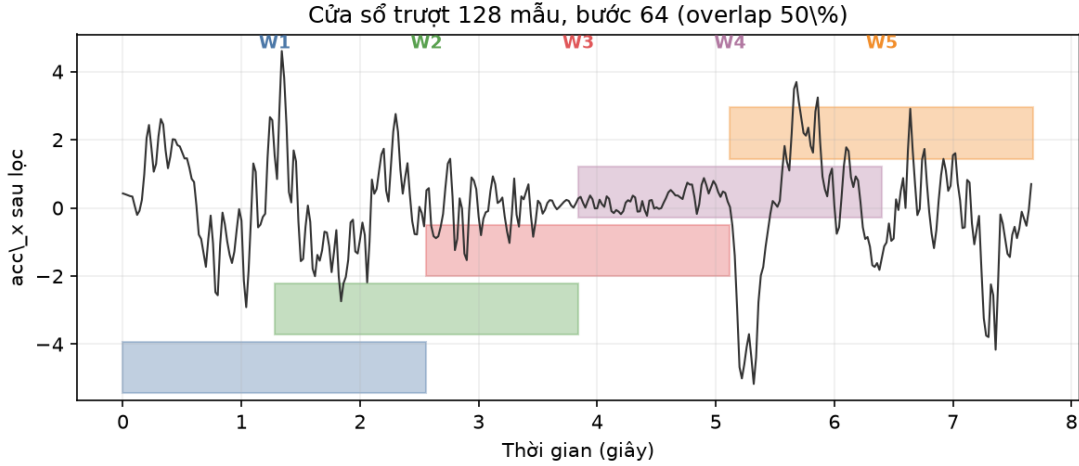
$$X \in \mathbb{R}^{39018 \times 128 \times 6}$$

Hình 1 minh họa tác dụng của bước lọc tín hiệu trên một đoạn dữ liệu walking của `userA`. Tín hiệu thô (đường xám) chứa nhiều cao tần và thành phần trọng lực một chiều; sau khi lọc low-pass 20 Hz và high-pass 0.3 Hz, tín hiệu (đường màu) mượt hơn và dao động quanh gốc do thành phần trọng lực đã bị tách khỏi ba trục gia tốc.



Hình 1: Tín hiệu gia tốc và vận tốc góc trước (xám) và sau (màu) bước tiền xử lý lọc Butterworth

Hình 2 minh họa cách chia cửa sổ trượt: mỗi cửa sổ dài 128 mẫu (2.56 giây) và bước trượt 64 mẫu, nên hai cửa sổ liên tiếp chồng lấn 50%. Cách này giữ tính liên tục của tín hiệu giữa các cửa sổ và tăng số mẫu huấn luyện, nhưng đồng thời tạo ra các cửa sổ gần giống nhau trong cùng một file, là lý do cần chia dữ liệu theo file ở phần sau.



Hình 2: Cửa sổ trượt 128 mẫu với bước 64 (overlap 50%) trên tín hiệu đã lọc

2.3 Cách chia dữ liệu

Sau tiền xử lý, tập inertial có 39018 cửa sổ, mỗi cửa sổ có kích thước (128, 6). 15 người dùng trong dữ liệu gồm userA, userB, userC, userD, userE, userF, userG, userH, userI, userK, userL, userM, userN, userP, userQ. Vì là bài toán closed-set, cả 15 người dùng đều tham gia huấn luyện và đánh giá; việc chia dữ liệu được thực hiện ở mức file để phục vụ cross-validation.

Bảng 3: Quy mô dữ liệu closed-set 15 người dùng

Tập	Users	Windows	Groups/files
Closed-set	15	39018	154

Với toàn bộ 15 người dùng, nhóm dùng *StratifiedGroupKFold*. Điểm quan trọng là **không chia ngẫu nhiên theo cửa sổ**. Vì sliding window tạo ra nhiều cửa sổ liên tiếp, các cửa sổ gần nhau trong cùng một file có nội dung tín hiệu rất giống nhau. Nếu random split theo cửa sổ, train và validation có thể cùng chứa các đoạn gần như trùng từ một phiên ghi, làm mô hình học đặc trưng của phiên ghi thay vì học hành vi người dùng. Đây là dạng data leakage và thường làm điểm đánh giá cao giả tạo.

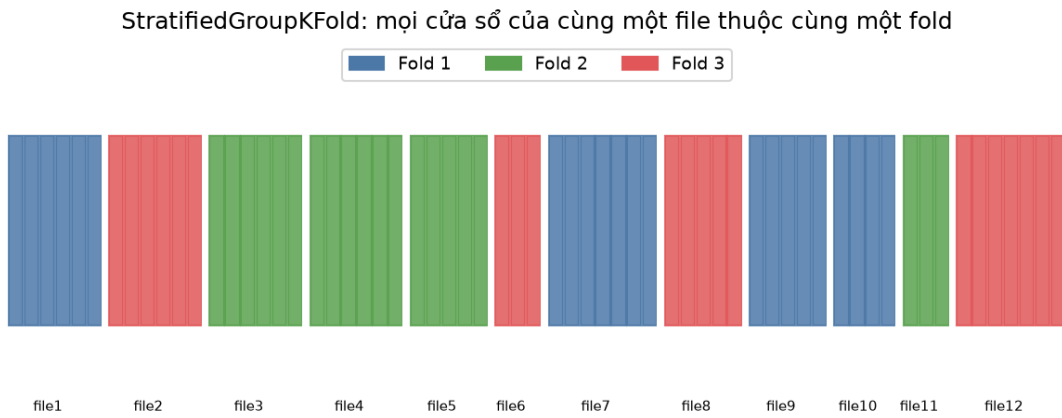
Để tránh leakage, mỗi cửa sổ được gán group có dạng `user__file`. Khi chia fold, toàn bộ cửa sổ sinh ra từ cùng một file luôn nằm trọn trong train hoặc trọn trong validation. Như vậy validation mô phỏng một phiên ghi mới hơn, khó hơn và gần thực tế hơn so với random split theo cửa sổ. Thành phần *stratified* giúp phân phối user trong các fold tương đối cân bằng, còn *group* đảm bảo rằng buộc không tách cùng một file sang hai tập. Tham số `shuffle=True`, `random_state=42` được dùng để kết quả có thể tái lập.

Nhóm chọn 3-fold vì số file giữa các user không đều; user ít file nhất là userQ, chỉ có 3 file inertial. Nếu dùng 5-fold, một số fold validation có nguy cơ thiếu class, khiến macro-F1 không còn phản ánh đủ 15 người dùng. Với 3-fold, mỗi fold vẫn có đủ lớp để đánh giá ổn định hơn.

Bảng 4: Thống kê train/validation của 3 fold closed-set

Fold	Train windows	Val windows	Train groups	Val groups
1	26207	12811	102	52
2	25585	13433	103	51
3	26244	12774	103	51

Hình 3 minh họa nguyên tắc chia: mỗi ô là một cửa sổ, các ô cùng màu thuộc cùng một fold, và ranh giới trắng phân tách các file. Toàn bộ cửa sổ sinh ra từ một file luôn nằm trọn trong một fold, nên không có cửa sổ nào của cùng một phiên ghi bị tách sang cả train lẫn validation.



Hình 3: Nguyên tắc StratifiedGroupKFold: các cửa sổ của cùng một file (group) luôn thuộc cùng một fold

2.4 Mô hình Random Forest

Random Forest là mô hình ensemble gồm nhiều cây quyết định, phù hợp với dữ liệu dạng bảng có số chiều vừa phải. Mỗi cửa sổ cảm biến được chuyển thành vector 65 đặc trưng thủ công. Cấu trúc feature gồm $6 \times 9 = 54$ đặc trưng thống kê/FFT cho 6 kênh, 9 đặc trưng correlation giữa các cặp trục được chọn, và 2 đặc trưng Signal Magnitude Area (SMA) cho accelerometer và gyroscope.

- Thống kê theo từng trục: mean, standard deviation, MAD, min, max, IQR, energy.
- Đặc trưng miền tần số: FFT entropy, dominant frequency.
- Tương quan giữa các trục.
- Signal Magnitude Area cho accelerometer và gyroscope.

Mô hình sử dụng `RandomForestClassifier` với 300 cây. Tham số cân bằng lớp là `class_weight=balanced_subsample`. Random Forest không yêu cầu chuẩn hoá đặc trưng và ít nhạy với khác biệt thang đo giữa các đặc trưng thống kê; cơ chế bagging trên 300 cây giảm phương sai dự đoán khi số mẫu mỗi người dùng không đều. Tham số `class_weight=balanced_subsample` tính lại trọng số lớp trên từng mẫu bootstrap, bù cho mất cân bằng số cửa sổ giữa các người dùng.

Để tránh leakage ở bước chuẩn hoá, `StandardScaler` chỉ được fit trên train fold. Sau đó scaler này mới được dùng để transform validation fold. Quy trình này đảm bảo validation không tham gia vào việc ước lượng trung bình và độ lệch chuẩn của đặc trưng.

2.5 Mô hình SVM (RBF)

SVM với kernel RBF được thêm như mô hình thứ ba, dùng **đúng 65 đặc trưng thủ công** như Random Forest. Lý do chọn SVM RBF là để bổ sung một góc nhìn khác về bản chất: trong khi Random Forest là mô hình dạng cây chia không gian theo các ngưỡng trục, SVM RBF là mô hình *margin-based* tìm siêu phẳng phân tách với biên lớn nhất trong không gian đặc trưng được ánh xạ phi tuyến. So sánh hai mô hình cùng đầu vào nhưng khác nguyên lý giúp kiểm chứng xem kết quả tốt đến từ chất lượng đặc trưng hay từ một loại mô hình cụ thể.

Cấu hình: `SVC(kernel="rbf", C=10, gamma="scale", class_weight="balanced")`.

- **Kernel RBF**: cho phép tạo biên quyết định phi tuyến, phù hợp khi ranh giới giữa các người dùng trong không gian 65 chiều không tuyến tính.
- **C = 10**: tham số phạt lỗi; giá trị vừa phải này ưu tiên phân loại đúng tập huấn luyện hơn mức mặc định $C = 1$ nhưng vẫn giữ regularization để tránh overfit.
- **gamma="scale"**: tự co giãn theo phương sai đặc trưng, tránh phải dò tay.
- **class_weight="balanced"**: bù cho việc số cửa sổ giữa các người dùng không đều, tương tự `balanced_subsample` của Random Forest.

Khác với Random Forest, SVM RBF **bắt buộc** chuẩn hoá đặc trưng vì kernel RBF tính khoảng cách Euclid giữa các mẫu; nếu một đặc trưng có thang đo lớn (ví dụ energy) sẽ lấn át các đặc trưng nhỏ. Vì vậy `StandardScaler` cũng chỉ fit trên train fold rồi áp lên validation, giống quy trình của Random Forest để so sánh công bằng.

2.6 Mô hình CNN 1D

CNN 1D được dùng để kiểm tra khả năng học trực tiếp từ chuỗi cảm biến. Sau khi đi qua `InertialDataset`, dữ liệu đầu vào có dạng $(B, 6, 128)$, trong đó B là batch size, 6 là số kênh cảm biến, 128 là chiều thời gian. `Conv1D` trượt kernel theo trục thời gian để học các mẫu biến thiên ngắn hạn của accelerometer và gyroscope.

```
Input: (6, 128)
Conv1D(6 -> 32, kernel=7) + BatchNorm + ReLU + MaxPool
Conv1D(32 -> 64, kernel=5) + BatchNorm + ReLU + MaxPool
```

```
Conv1D(64 -> 128, kernel=3) + BatchNorm + ReLU
AdaptiveAvgPool1D
Dropout(0.3)
Linear(128 -> 15)
```

Trong kiến trúc này, BatchNorm giúp ổn định phân phối activation trong quá trình huấn luyện; ReLU tạo phi tuyến; MaxPool giảm chiều thời gian và giữ mẫu nổi bật; AdaptiveAvgPool gom toàn bộ chuỗi thành một vector cố định trước khi phân loại; Dropout giảm overfit. Kiến trúc được giới hạn ở 3 tầng convolution để số tham số tương xứng với quy mô dữ liệu (39018 cửa sổ), tránh overfit khi mô hình quá lớn so với lượng mẫu huấn luyện.

Cấu hình huấn luyện:

- Optimizer: Adam.
- Learning rate: 0.001.
- Loss: CrossEntropy có class weight nghịch tần suất.
- Epoch: 40.
- Batch size: 128.
- Không dùng scheduler, không dùng MixUp/CutMix, không dùng sweep nhiều phase.

Cấu hình không dùng scheduler hay augmentation nhằm cô lập đóng góp của kiến trúc CNN cơ sở: kết quả phản ánh năng lực của mô hình base mà không bị ảnh hưởng bởi các kỹ thuật tăng cường, giúp so sánh trực tiếp với hai mô hình cổ điển trên cùng điều kiện huấn luyện tối giản.

3 Kết quả thí nghiệm đánh giá hiệu năng

3.1 Chỉ số đánh giá

Nhóm sử dụng nhiều chỉ số vì mỗi chỉ số phản ánh một khía cạnh khác nhau của hệ thống:

- **Accuracy:** tỷ lệ dự đoán đúng trên toàn bộ mẫu. Khi số cửa sổ giữa các người dùng không đều, accuracy bị chi phối bởi các lớp có nhiều mẫu hơn.
- **Macro-F1:** tính F1 riêng cho từng người dùng rồi lấy trung bình không trọng số. Mỗi người dùng đóng góp ngang nhau vào điểm cuối, nên một người dùng ít dữ liệu bị dự đoán sai sẽ kéo điểm xuống rõ rệt. Do tập dữ liệu mất cân bằng giữa các lớp, macro-F1 được dùng làm chỉ số chính để xếp hạng mô hình.

Với C class người dùng, macro-F1 được tính theo:

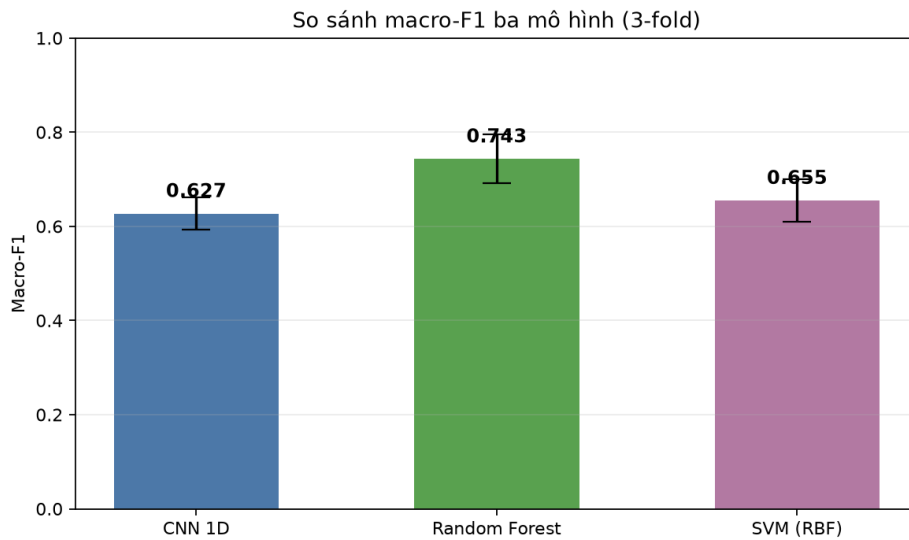
$$MacroF1 = \frac{1}{C} \sum_{c=1}^C F1_c$$

3.2 Kết quả tổng thể

Kết quả được chạy lại với cấu hình đơn giản, lưu tại `simple_results.json`. Bảng 5 trình bày điểm trung bình trên 3 fold.

Bảng 5: Kết quả closed-set theo 3-fold StratifiedGroupKFold

Mô hình	Macro-F1 trung bình	Độ lệch chuẩn	Accuracy trung bình
CNN 1D simple	0.6272	0.0336	0.6523
Random Forest	0.7434	0.0517	0.7482
SVM (RBF)	0.6550	0.0456	0.6740



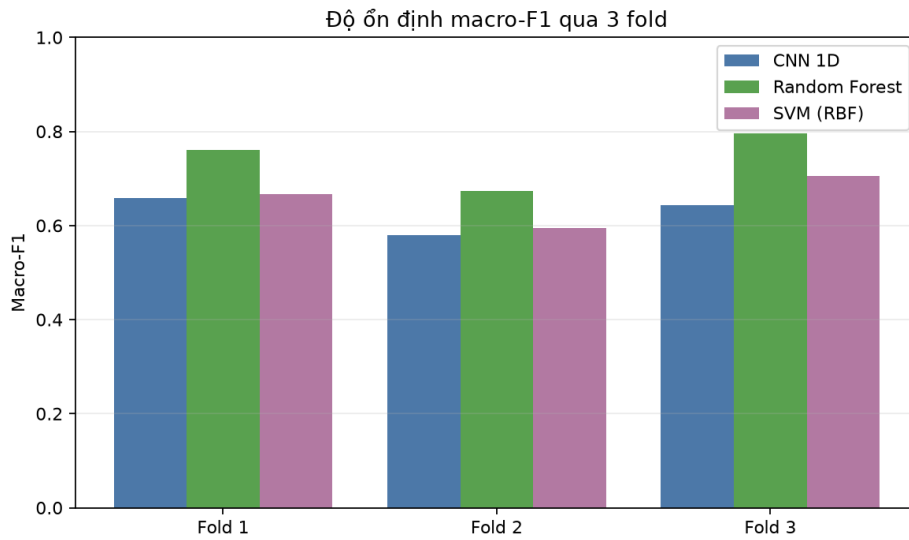
Hình 4: So sánh macro-F1 giữa CNN 1D, Random Forest và SVM (RBF)

Từ kết quả ở Bảng 5, có thể thấy mô hình Random Forest đạt hiệu năng vượt trội nhất với Macro-F1 trung bình là 0.7434 và Accuracy trung bình là 0.7482. SVM (RBF) đứng thứ hai với Macro-F1 trung bình 0.6550. Trong khi đó, mô hình học sâu CNN 1D đạt kết quả thấp nhất với Macro-F1 trung bình là 0.6272. Sự vượt trội của Random Forest so với CNN 1D cho thấy đối với tập dữ liệu có kích thước vừa phải và số lượng phiên ghi của mỗi người dùng còn hạn chế, các đặc trưng thống kê được thiết kế thủ công (hand-crafted features) vẫn mang lại độ chính xác và độ ổn định cao hơn so với việc để mô hình deep learning tự học các biểu diễn từ chuỗi tín hiệu thô.

3.3 Độ ổn định theo fold

Bảng 6: Macro-F1 trên từng fold

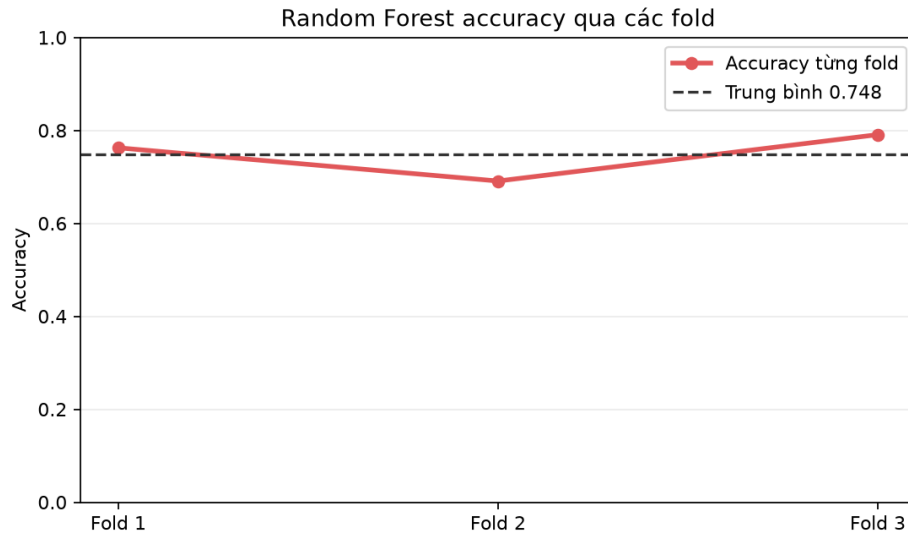
Mô hình	Fold 1	Fold 2	Fold 3
CNN 1D simple	0.6578	0.5804	0.6436
Random Forest	0.7607	0.6732	0.7962
SVM (RBF)	0.6660	0.5944	0.7046



Hình 5: Macro-F1 của từng mô hình qua 3 fold

Bảng 6 và Hình 5 thể hiện hiệu năng của các mô hình qua từng fold đánh giá. Random Forest cho thấy sự ổn định tốt nhất, mặc dù điểm Macro-F1 ở Fold 2 giảm xuống còn 0.6732 so với hai fold còn lại (đều trên 0.76). Cả ba mô hình đều đạt kết quả thấp nhất ở Fold 2 (CNN giảm xuống 0.5804, SVM xuống 0.5944), điều này chỉ ra rằng tập dữ liệu ở Fold 2 có thể chứa nhiều biến động hành vi phức tạp hơn hoặc có sự khác biệt lớn về phân phối dữ liệu giữa tập huấn luyện và kiểm định so với các fold khác.

3.4 Accuracy của Random Forest

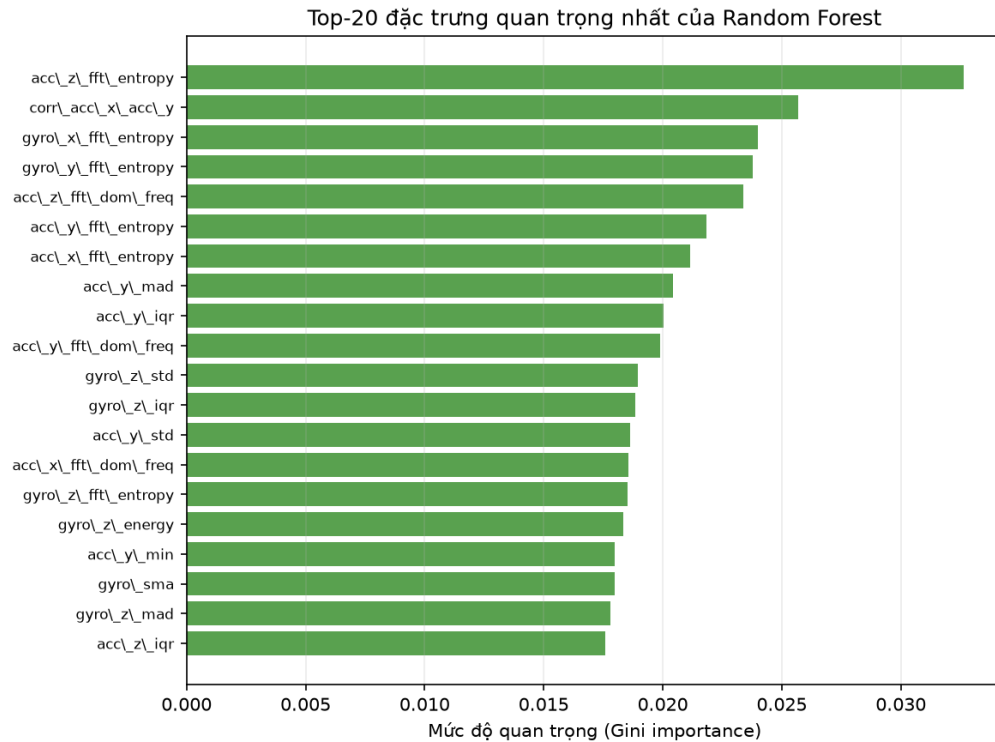


Hình 6: Accuracy của Random Forest qua từng fold

Accuracy trung bình của Random Forest đạt 0.7482, cao hơn macro-F1 (0.7434) một khoảng nhỏ. Chênh lệch này cho thấy mô hình dự đoán các người dùng có nhiều cửa sổ tốt hơn so với các người dùng ít dữ liệu; macro-F1 vì vậy phản ánh sát hơn hiệu năng trên toàn bộ 15 lớp khi dữ liệu không cân bằng.

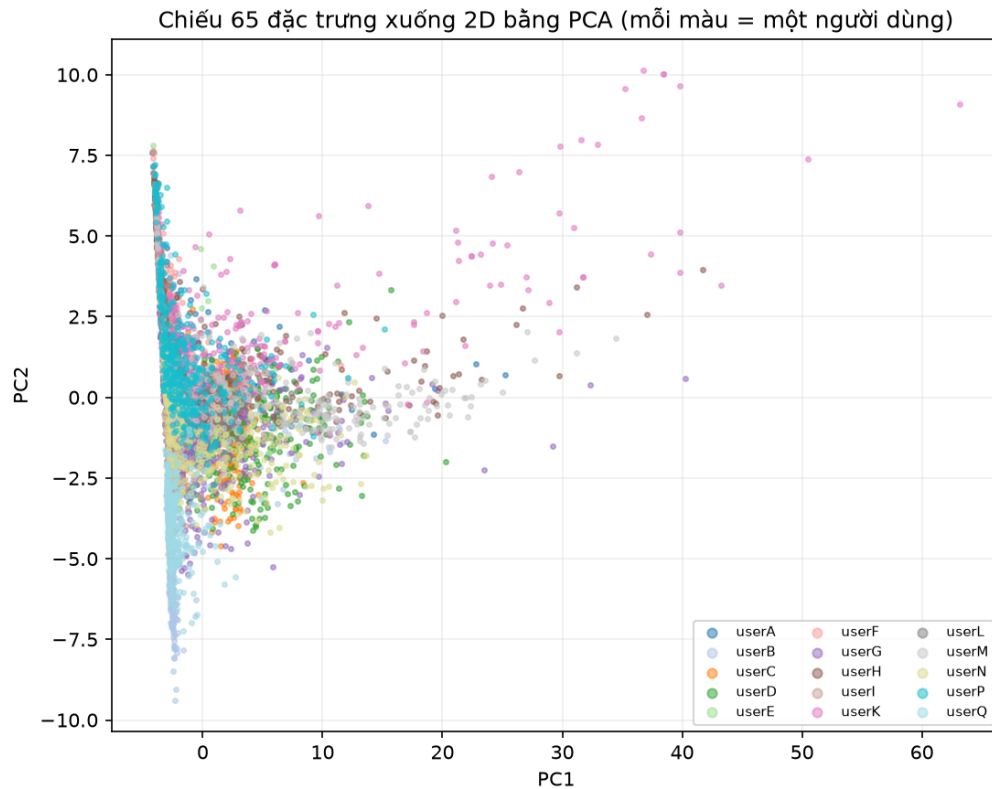
3.5 Phân tích mô hình Random Forest

Để hiểu mô hình chính dựa vào tín hiệu nào, Hình 7 liệt kê 20 đặc trưng có mức độ quan trọng (Gini importance) cao nhất do Random Forest tính ra. Các đặc trưng dẫn đầu chủ yếu thuộc nhóm thống kê biên độ và năng lượng của các trục gia tốc và con quay, cho thấy đặc trưng phân biệt người dùng nằm phần lớn ở mức độ và cường độ chuyển động, hơn là ở các đặc trưng tần số.



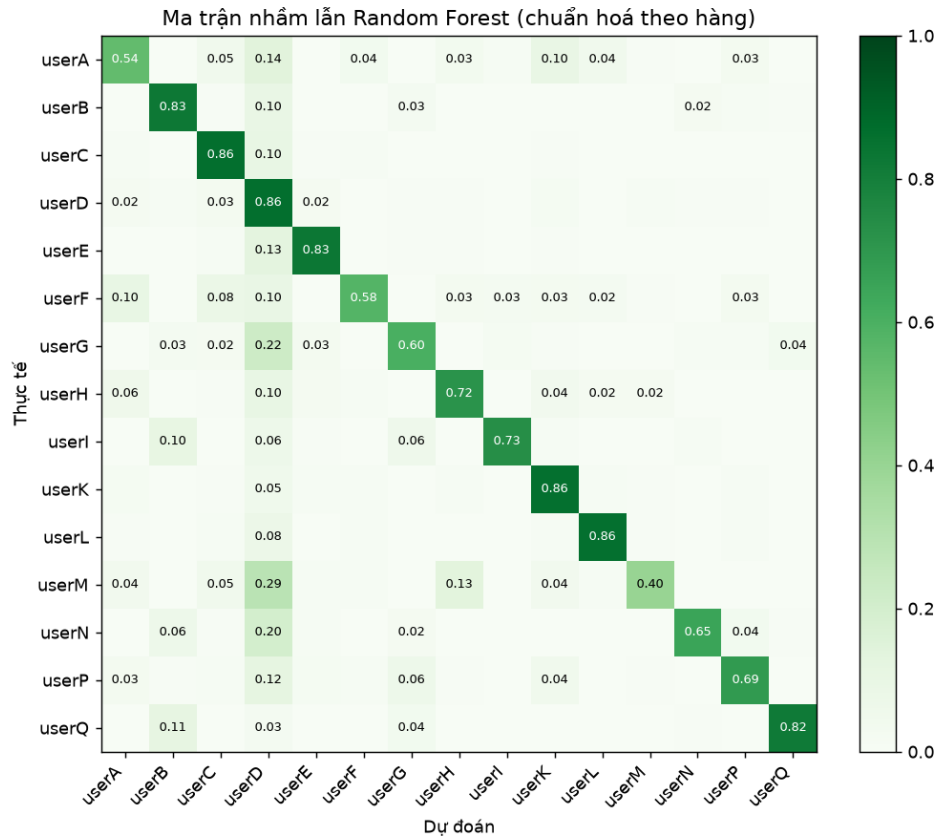
Hình 7: Top-20 đặc trưng quan trọng nhất theo Gini importance của Random Forest

Hình 8 chiếu 65 đặc trưng xuống 2 thành phần chính (PCA), tô màu theo người dùng. Một số người dùng tạo thành cụm tách biệt, nhưng nhiều cụm chồng lẫn nhau, phản ánh trực quan vì sao bài toán không đạt độ chính xác tuyệt đối: trong không gian đặc trưng, hành vi của một số người dùng tương đối giống nhau. PCA chỉ giữ hai chiều phương sai lớn nhất nên mức chồng lẫn thực tế mà mô hình xử lý trong không gian 65 chiều thấp hơn hình minh họa.



Hình 8: Phân bố các cửa sổ trong không gian 2 thành phần chính (PCA) của 65 đặc trưng, tô màu theo người dùng

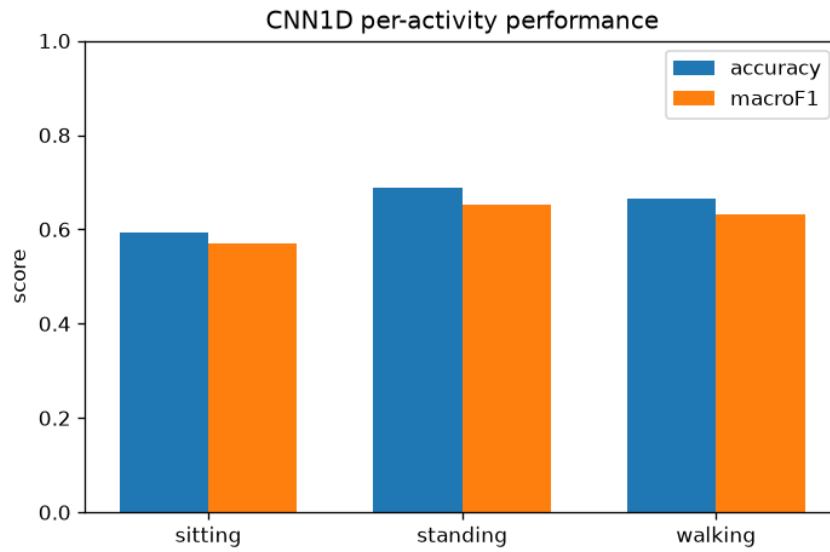
Hình 9 là ma trận nhầm lẫn chuẩn hoá theo hàng của Random Forest, gom dự đoán out-of-fold trên cả 3 fold. Đường chéo đậm cho thấy phần lớn cửa sổ được phân đúng người dùng; các ô lệch khỏi đường chéo chỉ ra những cặp người dùng dễ bị nhầm với nhau, thường là các cặp có biên độ chuyển động tương tự.



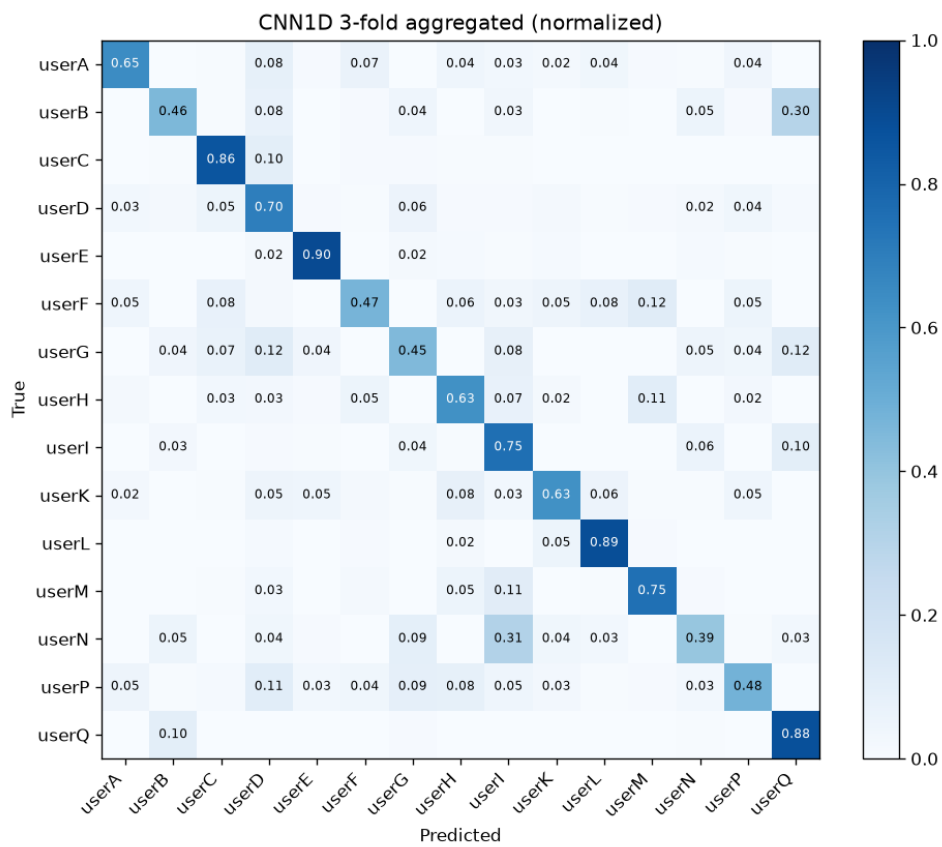
Hình 9: Ma trận nhầm lẫn chuẩn hoá của Random Forest trên 15 người dùng (gom 3 fold)

3.6 Phân tích CNN theo hoạt động

Pipeline đánh giá tổng hợp (`python -m src.evaluate`) còn sinh biểu đồ hiệu năng theo hoạt động và ma trận nhầm lẫn cho CNN. Hai hình này dùng để phân tích sâu hơn các lỗi của mô hình CNN trên 15 người dùng.



Hình 10: Hiệu năng CNN theo hoạt động sitting, standing, walking



Hình 11: Ma trận nhầm lẫn chuẩn hoá của CNN

4 Các chức năng chính của hệ thống và cách sử dụng

4.1 Chức năng chính

Hệ thống được tổ chức thành các bước độc lập, có thể chạy lại từng phần.

Bảng 7: Các chức năng chính của hệ thống

STT	Lệnh	Chức năng
1	<code>python-msrc.io</code>	Quét dữ liệu thô, parse tên file, sinh <code>manifest.csv</code> .
2	<code>python-msrc.preprocess</code>	Tiền xử lý cảm biến, lọc tín hiệu, chia cửa sổ, sinh <code>windows.npz</code> .
3	<code>python-msrc.features</code>	Trích đặc trưng gõ phím phục vụ phân tích bổ sung.
4	<code>python-mscripts.run_simple_pipeline</code>	Chạy pipeline đơn giản gồm CNN 1D, RF và SVM, lưu <code>simple_results.json</code> .
5	<code>python-msrc.evaluate</code>	Gom prediction CNN, vẽ ma trận nhầm lẫn và per-activity, chạy RF/SVM, lưu <code>evaluate_results.json</code> .
6	<code>python-mscripts.plot_simple_results</code>	Vẽ biểu đồ so sánh mô hình, fold và accuracy.
7	<code>jupyterlab</code> <code>simple_pipeline.ipynb</code>	Chạy notebook minh họa kết quả và biểu đồ trong JupyterLab.

4.2 Cài đặt môi trường

Yêu cầu Python 3.10 trở lên. Các gói chính được liệt kê trong `requirements.txt`: `numpy`, `pandas`, `scipy`, `scikit-learn`, `torch`, `matplotlib`, `seaborn`, `joblib`, `tqdm`, `jupyterlab`.

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt --index-url https://pypi.org/simple
```

4.3 Chạy pipeline đơn giản

```
source venv/bin/activate
python -m scripts.run_simple_pipeline
python -m scripts.plot_simple_results
```

Kết quả chính:

- `simple_results.json`: điểm số CNN và RF.
- `simple_model_comparison.png`: so sánh tổng thể.
- `simple_fold_macro_f1.png`: điểm từng fold.
- `simple_rf_accuracy.png`: accuracy của RF.

4.4 Chạy bằng JupyterLab

Nhóm đã chuẩn bị notebook `simple_pipeline.ipynb`. Mở notebook bằng:

```
source venv/bin/activate
jupyter lab simple_pipeline.ipynb
```

Trong notebook, biến `RUN_TRAIN` mặc định là `False` để không train lại CNN mỗi lần mở. Nếu muốn chạy lại toàn bộ, đổi `RUN_TRAIN = True`. Nếu chỉ muốn cập nhật RF và biểu đồ, đổi `RUN_RF_ONLY = True`.

5 Cấu trúc mã nguồn và vai trò các class/method chính

5.1 Tổ chức mã nguồn

Mã nguồn được tách theo đúng luồng xử lý: đọc dữ liệu, tiền xử lý, định nghĩa dataset, huấn luyện, đánh giá và vẽ hình. Trong báo cáo chỉ nêu tên file chính để tránh làm phần trình bày bị nặng bởi đường dẫn thư mục.

5.2 Vai trò các module chính

Bảng 8: Vai trò các module chính

File	Vai trò
<code>config.py</code>	Khai báo đường dẫn, kích thước cửa sổ, danh sách kênh cảm biến, danh sách hoạt động, seed.
<code>io.py</code>	Parse tên file, đọc CSV, ước lượng tần số lấy mẫu, xây dựng manifest.
<code>preprocess.py</code>	Resample, lọc Butterworth, tách trọng lực, chia sliding window.
<code>features.py</code>	Trích đặc trưng thống kê, FFT, correlation, SMA và đặc trưng keystroke.
<code>datasets.py</code>	Định nghĩa <code>InertialDataset</code> , chuẩn hoá dữ liệu, chuyển shape cho Conv1D.
<code>models.py</code>	Định nghĩa CNN1D và factory <code>make_model</code> .

File	Vai trò
<code>train.py</code>	Chạy huấn luyện 3-fold, lưu checkpoint và log experiment.
<code>evaluate.py</code>	Chạy Random Forest và SVM (RBF) baseline, gom prediction CNN, vẽ confusion/per-activity.
<code>run_simple_pipeline.py</code>	Script đơn giản hoá để chạy ba mô hình chính: CNN 1D, RF và SVM.
<code>plot_simple_results.py</code>	Tạo các biểu đồ chính dùng trong báo cáo.

5.3 Class và method quan trọng

- `InertialDataset.__getitem__`: lấy một cửa sổ cảm biến, chuẩn hoá theo mean/std của train fold, chuyển từ shape (T, C) sang (C, T) cho PyTorch Conv1D.
- `CNN1D.forward`: chạy chuỗi Conv1D, BatchNorm, ReLU, pooling và linear head để dự đoán người dùng.
- `run_cv`: thực hiện vòng lặp 3-fold, gọi train từng fold, lưu checkpoint và metric.
- `rf_baseline`: trích đặc trưng thủ công, chuẩn hoá, train Random Forest và tính macro-F1/accuracy.
- `svm_baseline`: dùng cùng 65 đặc trưng và quy trình chuẩn hoá như `rf_baseline`, train SVM RBF và tính macro-F1/accuracy.
- `plot_model_comparison`, `plot_fold_scores`, `plot_rf_accuracy`: sinh hình minh hoạ kết quả.

6 Khó khăn gặp phải và cách giải quyết

6.1 Tên file và schema dữ liệu không đồng nhất

Các file dữ liệu có nhiều quy ước đặt tên khác nhau, ví dụ `_s1_att2.csv`, `_s1_r3.csv`, hoặc thiếu một số thành phần. Nếu parse thủ công từng trường hợp, pipeline dễ lỗi và khó mở rộng. Nhóm xử lý bằng các regex riêng trong `io.py`, sau đó sinh manifest thống nhất.

6.2 Nguy cơ rò rỉ dữ liệu khi chia train/validation

Nếu chia ngẫu nhiên theo cửa sổ, các cửa sổ từ cùng một file có thể xuất hiện ở cả train và validation. Do cửa sổ trượt có overlap 50%, nhiều mẫu liền kề gần như cùng biểu diễn một đoạn chuyển động. Điều này làm điểm số tăng giả tạo vì mô hình học đặc trưng riêng của phiên ghi thay vì học hành vi người dùng. Giải pháp là dùng *StratifiedGroupKFold* theo group `user__file`, đảm bảo cùng một file không bị tách sang hai tập.

6.3 Ít file ở một số người dùng

Số file inertial giữa các user không đều. User ít nhất là `userQ`, chỉ có 3 file inertial. Nếu cố dùng 5-fold, một số fold validation có thể thiếu user này, làm macro-F1 không đánh giá đủ class và khiến kết quả kém tin cậy. Giải pháp là chọn 3-fold, cân bằng giữa số lần đánh giá và điều kiện mỗi fold vẫn có đủ 15 người dùng.

6.4 Mất cân bằng số cửa sổ giữa các file

Một số file dài sinh ra rất nhiều cửa sổ, khiến một vài người dùng hoặc phiên ghi chi phối dữ liệu huấn luyện. Giải pháp là giới hạn tối đa 300 cửa sổ mỗi file và lấy mẫu đều theo thời gian bằng `np.linspace`.

6.5 Dữ liệu cảm biến có nhiễu và khác tần số

Dữ liệu thực tế từ điện thoại có thể lệch tần số lấy mẫu hoặc nhiễu cao tần. Pipeline xử lý bằng cách resample về 50 Hz, low-pass 20 Hz và high-pass 0.3 Hz cho gia tốc.

6.6 CNN không vượt Random Forest

Kết quả cho thấy Random Forest đạt macro-F1 0.7434, cao hơn CNN 1D (0.6272) khoảng 0.116. Với tập 39018 cửa sổ và số phiên ghi giới hạn cho mỗi người dùng, CNN chưa đủ dữ liệu để học biểu diễn vượt qua bộ 65 đặc trưng thống kê đã mã hoá sẵn thông tin về biên độ, năng lượng, tần số và tương quan giữa các trục. Đây là hiện tượng thường gặp khi áp dụng học sâu trên tập dữ liệu cỡ vừa và nhỏ.

7 Khám phá mới hoặc kết luận

7.1 Khám phá chính

1. **Random Forest đạt hiệu năng cao nhất trong ba mô hình.** Với macro-F1 0.7434, Random Forest vượt SVM RBF (0.6550) và CNN 1D (0.6272) trên cùng điều kiện đánh giá, nên được chọn làm mô hình chính của báo cáo.
2. **Đánh giá group-by-file là bắt buộc.** Nếu không nhóm theo file, kết quả có thể bị leakage và không phản ánh khả năng tổng quát hoá tới phiên ghi mới.
3. **Hai mô hình trên cùng đặc trưng cho kết quả gần nhau.** Random Forest và SVM RBF dùng chung 65 đặc trưng và cho điểm tương đồng, cho thấy phần lớn hiệu năng đến từ chất lượng đặc trưng thủ công chứ không phải từ một loại mô hình cụ thể.
4. **Dữ liệu chuyển động giàu thông tin hơn khi hoạt động có chuyển động rõ.** Các hoạt động như walking/standing thường cho tín hiệu phân biệt người dùng tốt hơn sitting.

5. **Deep learning không phải lúc nào cũng thắng.** Trong bối cảnh dữ liệu nhỏ, mô hình cổ điển với feature tốt có thể vượt mô hình CNN đơn giản.

7.2 Kết luận

Đồ án đã hoàn thành pipeline từ dữ liệu thô tới đánh giá mô hình: đọc CSV, chuẩn hoá manifest, tiền xử lý cảm biến, chia cửa sổ, trích đặc trưng, huấn luyện ba mô hình (Random Forest, SVM RBF, CNN 1D), đánh giá closed-set 15 người dùng và sinh biểu đồ báo cáo.

- Random Forest: mô hình chính, đạt macro-F1 0.7434 và accuracy 0.7482.
- SVM (RBF): baseline thứ hai trên cùng đặc trưng, đạt macro-F1 0.6550.
- CNN 1D: mô hình học sâu bổ sung, đạt macro-F1 0.6272.

7.3 Hướng phát triển

- Thu thêm dữ liệu nhiều phiên hơn cho mỗi người dùng.
- Kết hợp inertial với keystroke ở mức feature fusion.
- Thử thêm đặc trưng hoặc tinh chỉnh siêu tham số cho CNN để rút ngắn khoảng cách với mô hình cổ điển.
- Tạo giao diện demo đơn giản để upload CSV và dự đoán người dùng.

7.4 Đóng gói sản phẩm nộp

- Mã nguồn dự án và cấu hình thí nghiệm.
- Hướng dẫn cài đặt và chạy: `README.md` và `requirements.txt`.
- Báo cáo cuối: `bao_cao.pdf`.
- Notebook minh hoạ: `simple_pipeline.ipynb`.
- Kết quả số: `simple_results.json`.
- Các hình kết quả dùng trong báo cáo.

Tài liệu

- [1] Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 2011.
- [2] Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.

- [3] Virtanen et al., *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*, Nature Methods, 2020.
- [4] Anguita et al., *A Public Domain Dataset for Human Activity Recognition Using Smartphones*, ESANN, 2013.